CoinKeeper

System Architecture Report

Course Information

EECE 430
Advanced Software Engineering

Instructor: **Dr. Khaled Dassouki**

Team Members

Ziad Abillama

Hamed Bou Saleh

William Nader

A microservices-based personal finance tracker
deployed on Kubernetes

November 18, 2025

Contents

1 Executive Summary **3**

2 Introduction **3**

3 Microservices Architecture **3**

3.1 Why Microservices? Benefits and Tradeoffs 4

3.1.1 Benefits 4

3.1.2 Tradeoffs and Challenges 4

3.1.3 Why CoinKeeper Benefits Despite These Tradeoffs 5

4 System Architecture Overview **5**

4.1 High-Level Flow 5

4.2 Logical Architecture Diagram 5

4.3 Deployment Architecture Diagram 6

4.4 Routing by Path 7

5 Sequence Diagrams **7**

5.1 User Login Flow 7

5.2 Dashboard Data Fetch 7

5.3 Adding an Expense 8

6 Security Architecture **8**

6.1 Authentication and Authorization 9

6.2 Secrets Management 9

6.3 Network Security 10

6.4 CORS Configuration 10

6.5 Input Validation 10

6.6 Future Security Enhancements 10

7 Architecture Components **11**

7.1 Frontend Layer 11

7.2 Backend Layer 11

7.3 Data Layer (MongoDB) 11

7.4 Infrastructure Layer (Kubernetes) 11

8 Kubernetes Deployment Model **12**

8.1 Deployments and Replicas 12

8.2 Services and Internal Networking 12
8.3 Ingress 12

9 Project Structure **13**

10 Architectural Benefits **14**

11 Conclusion **14**

1 Executive Summary

CoinKeeper is a modern, production-ready personal finance tracking application that demonstrates advanced software engineering principles through a microservices architecture deployed on Kubernetes. This report provides a comprehensive analysis of the system's architecture, design decisions, and implementation strategies.

The application separates concerns into five independent microservices (Auth, Expenses, Budgets, Analytics, and Frontend), each deployable and scalable independently. Through Kubernetes orchestration and NGINX Ingress routing, CoinKeeper achieves high availability, fault isolation, and cloud-native deployment patterns suitable for enterprise-grade applications.

2 Introduction

CoinKeeper is designed as a modern, scalable, cloud-ready personal finance platform. The system follows a **microservices architecture**, where each core domain of the application is implemented as an independent service that can be developed, deployed, and scaled separately.

By containerizing each service and deploying them on **Kubernetes**, CoinKeeper achieves:

- clear separation of concerns between authentication, expenses, budgets, analytics, and UI,
- independent scaling and deployment of services,
- fault isolation (one failing service does not bring down the entire app),
- a production-style, cloud-native setup suitable for real workloads.

High-level idea: the frontend talks to a single URL, while an Ingress controller routes requests to multiple backend microservices running inside the cluster. All business data is stored in MongoDB.

3 Microservices Architecture

CoinKeeper is split into the following microservices:

1. **Auth Service** (`auth-service`)
   Handles user registration, login, and JWT token generation/verification. Centralizes user management and security concerns.
2. **Expenses Service** (`expenses-service`)
   Responsible for CRUD operations on expenses: creating, listing, updating, and deleting user expenses.
3. **Budgets Service** (`budgets-service`)
   Manages user budgets per category (e.g. Food, Rent). Computes how much of each budget has been consumed.
4. **Analytics Service** (`analytics-service`)
   Computes higher-level summaries and insights: total spent, total remaining, per-category breakdowns, and trends over time.

5. **Frontend Service** (`frontend`)
   A React single-page application, built into static files and served via NGINX. Provides the CoinKeeper dashboard UI (dark theme, cards, summaries, etc.).

3.1 Why Microservices? Benefits and Tradeoffs

3.1.1 Benefits

- **Independent Scalability:** Heavy services such as analytics can be given more replicas without over-provisioning simpler services like authentication.
- **Fault Isolation:** If the analytics service crashes, users can still log in and record expenses. Service failures are contained.
- **Technology Flexibility:** Future services could be written in other languages (Python for ML, Go for high-performance) while still communicating over HTTP.
- **Independent Deployments:** Each service can be updated or rolled back without redeploying the others, enabling continuous delivery.
- **Team Autonomy:** Different teams can own different services with clear boundaries and responsibilities.
- **Maintainability:** Smaller codebases are easier to understand, test, and extend.

3.1.2 Tradeoffs and Challenges

While microservices offer significant benefits, they also introduce complexity:

- **Operational Overhead:** Managing multiple services requires more infrastructure (Kubernetes, service mesh, monitoring) compared to a monolithic application.
- **Distributed System Complexity:** Network failures, latency, and partial failures must be handled explicitly. Service-to-service communication adds complexity.
- **Debugging Challenges:** Tracing requests across multiple services requires distributed tracing tools (like Jaeger or Zipkin). Logs are scattered across services.
- **Network Latency:** Inter-service communication over HTTP introduces latency compared to in-process function calls.
- **Data Consistency:** Maintaining consistency across services sharing data (like users) requires careful design. ACID transactions across services are difficult.
- **Testing Complexity:** Integration testing requires running multiple services. End-to-end tests become more complex.
- **Deployment Complexity:** Coordinating deployments across multiple services requires orchestration and careful versioning.

3.1.3 Why CoinKeeper Benefits Despite These Tradeoffs

Despite the added complexity, CoinKeeper benefits from microservices because:

- **Clear Domain Boundaries:** Authentication, expenses, budgets, and analytics are naturally separate domains with minimal cross-cutting concerns.
- **Scalability Requirements:** Analytics may require more resources during reporting, while auth remains lightweight. Independent scaling is valuable.
- **Learning and Demonstration:** As an educational project, it demonstrates industry-standard architectural patterns used in production systems.
- **Future Extensions:** Adding new features (notifications, recommendations, third-party integrations) is easier without modifying core services.

4 System Architecture Overview

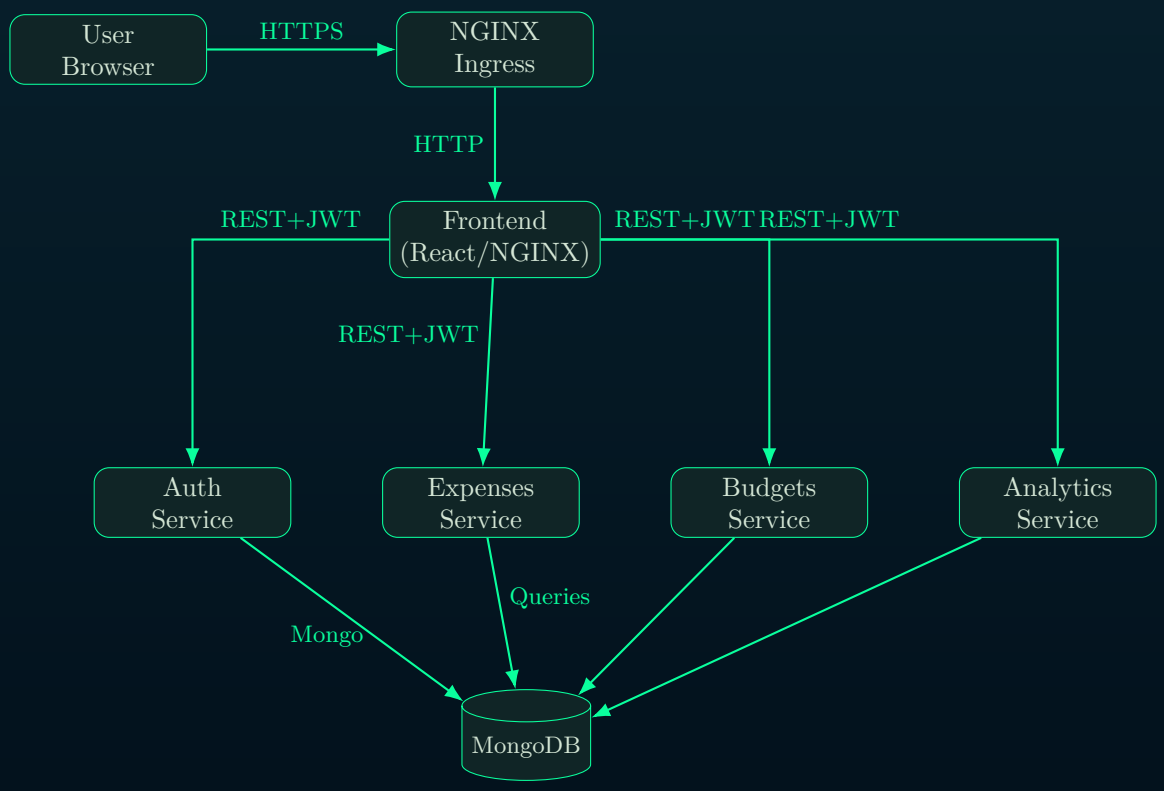
4.1 High-Level Flow

At a high level, CoinKeeper follows this flow:

User browser → NGINX Ingress Controller → Kubernetes Services → Pods (containers) → MongoDB.

4.2 Logical Architecture Diagram

The logical architecture shows the *what* — the components and their relationships:

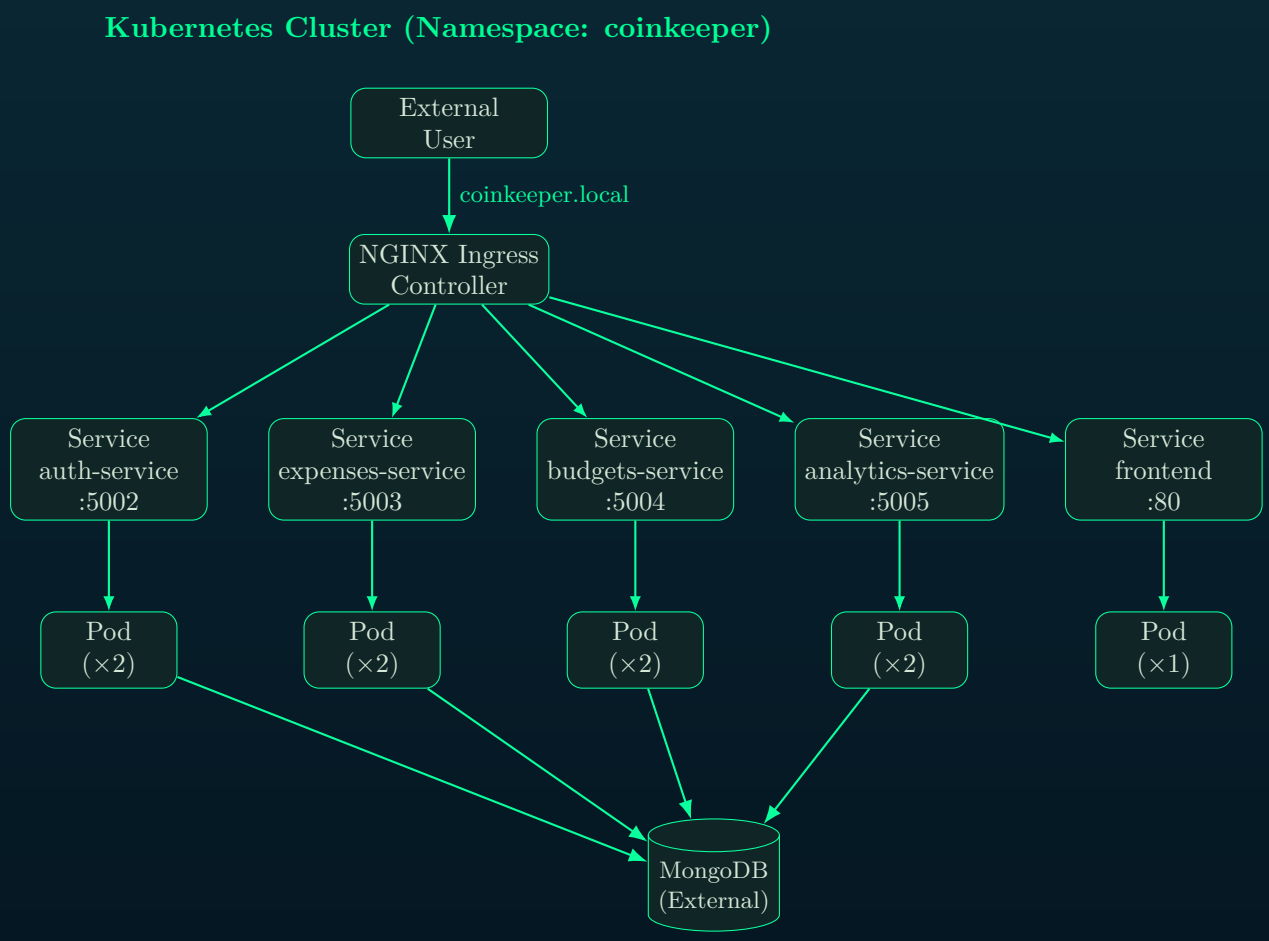


Legend – Logical Architecture

- **Boxes:** Services/components
- **Arrows:** Communication paths
- **HTTPS:** Encrypted external traffic
- **REST+JWT:** Authenticated API calls
- **Mongo Queries:** Database operations

4.3 Deployment Architecture Diagram

The deployment architecture shows the *where* — how components are deployed on Kubernetes:



Legend – Deployment Architecture

- **Cluster Box:** Kubernetes namespace boundary
- **Service:** ClusterIP service (internal DNS)
- **Pod (×N):** Container replicas
- **MongoDB:** External database (cloud-hosted)

4.4 Routing by Path

The NGINX Ingress controller receives all external HTTP traffic for the hostname `coinkeeper.local` and routes based on the request path:

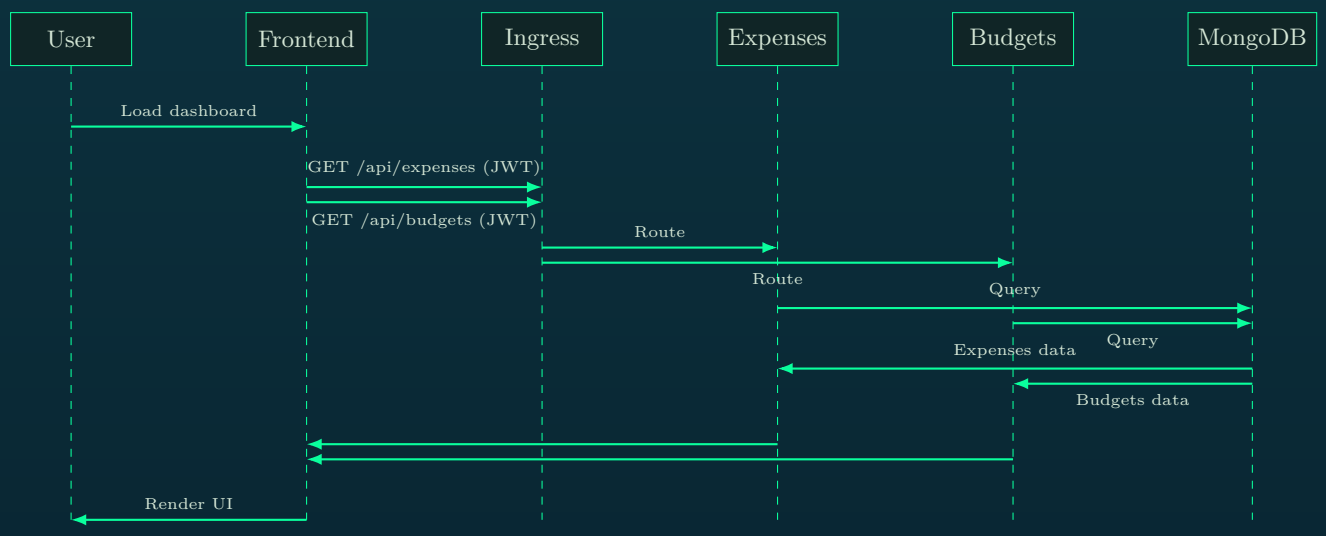
- `/` → **frontend** service (React UI)
- `/api/auth` → **auth-service**
- `/api/expenses` → **expenses-service**
- `/api/budgets` → **budgets-service**
- `/api/analytics` → **analytics-service**

The React application itself is configured to use a `baseURL` of `/api`, so from the UI perspective it simply calls relative endpoints such as `/api/auth/login` or `/api/expenses`.

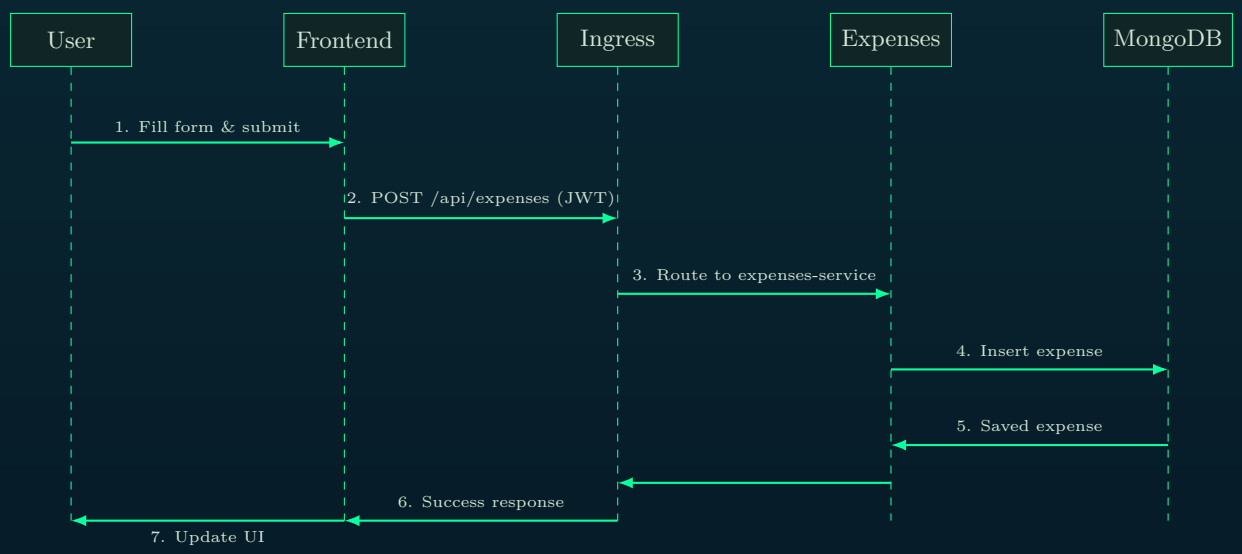
5 Sequence Diagrams

5.1 User Login Flow

5.2 Dashboard Data Fetch



5.3 Adding an Expense



6 Security Architecture

Security is a critical concern in any multi-service application. CoinKeeper implements multiple layers of security:

6.1 Authentication and Authorization

JWT-Based Authentication:

- Users authenticate once through the Auth Service
- Upon successful login, a signed JWT token is issued
- All subsequent requests include this JWT in the Authorization header
- Each microservice validates the JWT independently using a shared secret
- Tokens expire after 7 days, requiring re-authentication

Advantages of this approach:

- Stateless authentication — no session storage required
- Services can verify tokens without calling the auth service
- Reduced database load for auth checks
- Easy to scale horizontally

6.2 Secrets Management

Kubernetes Secrets:

- Sensitive data (MongoDB URI, JWT secret) stored in Kubernetes Secrets
- Secrets are base64-encoded and encrypted at rest (if etcd encryption is enabled)
- Injected into pods as environment variables
- Never hardcoded in source code or Docker images
- Access controlled via Kubernetes RBAC

Example secret usage:

```
envFrom:
  - secretRef:
      name: coinkeeper-backend-secrets
```

6.3 Network Security

HTTPS Termination:

- HTTPS terminates at the NGINX Ingress Controller
- All external traffic is encrypted
- Internal cluster traffic uses HTTP (within trusted network)
- Can be enhanced with service mesh (Istio/Linkerd) for mTLS between services

6.4 CORS Configuration

Cross-Origin Resource Sharing:

- CORS enabled on all backend services
- Restricts which origins can access the API
- Configured to allow requests from the frontend domain
- Prevents unauthorized cross-origin requests

6.5 Input Validation

Data Validation:

- Mongoose schema validation for all database models
- Express middleware validates request payloads
- Amount fields restricted to positive numbers
- Required fields enforced at API level
- Category and title fields trimmed to prevent whitespace attacks

6.6 Future Security Enhancements

Areas for improvement:

- **Rate Limiting:** Implement rate limiting at Ingress level to prevent DDoS attacks
- **API Gateway:** Add an API Gateway (Kong, Ambassador) for centralized auth and rate limiting
- **Service Mesh:** Deploy Istio or Linkerd for mutual TLS between services
- **Secret Rotation:** Implement automated secret rotation for JWT keys
- **Monitoring & Alerts:** Set up Prometheus/Grafana for security monitoring

- **Network Policies:** Kubernetes NetworkPolicies to restrict pod-to-pod communication

7 Architecture Components

7.1 Frontend Layer

- Implemented with **React**.
- Built into static assets and served by **NGINX** inside a container.
- Provides the dark-themed CoinKeeper dashboard with summary cards, budgets, and navigation (Dashboard, Expenses, Budgets, Analytics, etc.).
- Deployed to Kubernetes as a `frontend` Deployment and exposed internally via a ClusterIP Service.

7.2 Backend Layer

Each backend microservice is a Node.js application with its own:

- **Express routes** (e.g. `/api/auth`, `/api/expenses`),
- **Mongoose models** for MongoDB documents,
- **middleware** (notably JWT auth middleware),
- **Dockerfile** and **.env** configuration.

Common characteristics:

- Use a shared **MongoDB URI** and **JWT_SECRET** injected via Kubernetes Secrets.
- Communicate only via HTTP; there is no direct database sharing between services except through MongoDB.
- Use JSON Web Tokens to authenticate requests from the frontend.

7.3 Data Layer (MongoDB)

- All services connect to the same MongoDB instance.
- Collections are logically separated by domain (users, expenses, budgets, etc.).
- Connection details are stored centrally in the Kubernetes Secret `coinkeeper-backend-secrets`.

7.4 Infrastructure Layer (Kubernetes)

- **Namespace** `coinkeeper` isolates all resources for the app.
- **Deployments** manage desired replicas of each service.
- **Services (ClusterIP)** provide stable internal endpoints (e.g. `auth-service`, `expenses-service`).

- **Ingress Controller (NGINX)** exposes HTTP traffic externally and routes to the correct services.
- **Secrets** store sensitive values like `MONGODB_URI` and `JWT_SECRET`.

8 Kubernetes Deployment Model

8.1 Deployments and Replicas

Each backend service is deployed as a Kubernetes **Deployment** with **2 replicas**, for example:

- `auth-service` Deployment → 2 pods
- `expenses-service` Deployment → 2 pods
- `budgets-service` Deployment → 2 pods
- `analytics-service` Deployment → 2 pods

Replicas provide:

- **High availability** (one pod can fail without downtime),
- **Load distribution** across multiple pods,
- **Smooth rolling updates** during deployments.

8.2 Services and Internal Networking

Each Deployment has a corresponding **ClusterIP Service**. For example:

- `auth-service` (port 5002)
- `expenses-service` (port 5003)
- `budgets-service` (port 5004)
- `analytics-service` (port 5005)
- `frontend` (port 80)

Inside the cluster, services are reachable by DNS name, such as `http://auth-service:5002`. This completely decouples internal traffic from host ports.

8.3 Ingress

The **Ingress** resource `coinkeeper-ingress` (class `nginx`) binds the public host `coinkeeper.local` to the internal services:

- `/` → `frontend:80`

- `/api/auth` → `auth-service:5002`
- `/api/expenses` → `expenses-service:5003`
- `/api/budgets` → `budgets-service:5004`
- `/api/analytics` → `analytics-service:5005`

This gives the illusion of a single monolithic backend while preserving a microservices implementation behind the scenes.

9 Project Structure

The final project structure is:

```
CoinKeeper/
    services/
        auth-service/
        expenses-service/
        budgets-service/
        analytics-service/
    frontend/
    k8s/
        namespace.yaml
        auth.yaml
        expenses.yaml
        budgets.yaml
        analytics.yaml
        frontend.yaml
        ingress.yaml
```

- `/services` contains all backend microservices.
- `/frontend` contains the React application.
- `/k8s` contains the Kubernetes manifests used for deployment.

This mirrors the logical architecture: presentation, services, and infrastructure are clearly separated.

10 Architectural Benefits

Key advantages of the CoinKeeper architecture:

- **Independent scaling**: each microservice can be scaled horizontally according to its own load profile.
- **Fault tolerance**: failures are isolated to individual services; the rest of the app keeps working.
- **Maintainability**: services are small, focused, and have a single responsibility.
- **Independent deployments**: new features or bug fixes can be released service-by-service.
- **Performance optimization**: analytics or reporting can be given more resources without affecting other domains.
- **Cloud-native readiness**: running on Kubernetes aligns the project with real-world production practices.
- **Security isolation**: compromised services can be contained more easily.

11 Conclusion

CoinKeeper is not just a simple expense tracker; it is a full microservices-based, Kubernetes-deployed system that resembles the architecture of real production applications.

By separating the system into clearly-defined services (Auth, Expenses, Budgets, Analytics, Frontend) and managing them through Kubernetes Deployments, Services, Ingress, and Secrets, the application becomes:

- scalable,
- resilient,
- secure,
- and easy to evolve.

This architecture also provides an excellent foundation for future extensions such as notifications, reporting exports, or integration with external APIs without needing to redesign the entire system.

Future Enhancements:

- Service mesh (Istio) for advanced traffic management
- Distributed tracing (Jaeger/Zipkin)
- Centralized logging (ELK stack)
- CI/CD pipeline (GitLab CI, ArgoCD)

- Auto-scaling based on metrics
- Multi-region deployment